

Programming Assignment 04

Disk Scheduling and RAID-backed Swap in xv6-RISC-V

Course: CS3523 – Operating Systems

Institution: IIT Hyderabad

Semester: Spring 2025–26

Abstract

This report describes the design and implementation of PA4, which extends the xv6 teaching operating system with disk-backed swap, configurable disk scheduling policies (FCFS and SSTF), RAID-based virtual disk simulation, and per-process I/O statistics. The implementation builds directly on PA1 (system-call accounting), PA2 (MLFQ scheduling), and PA3 (page replacement and swap). Every design decision, data structure, and kernel modification is documented in the sections that follow.

Submitted in fulfilment of the requirements of CS3523

Contents

1	Introduction and Motivation	3
2	Background and Prior-Assignment Context	3
2.1	What Was Already in Place (PA1–PA3)	3
2.2	Disk I/O Path in Vanilla xv6	3
3	New Data Structures	4
3.1	<code>struct disk_req</code> – The Request Queue Entry	4
3.2	Extensions to <code>struct proc</code> (<code>proc.h</code>)	4
3.3	Scheduler State Variables	4
4	Disk Scheduling Subsystem (<code>disksched.c</code>)	5
4.1	Initialisation – <code>disksched_init</code>	5
4.2	Policy Selection – <code>disksched_set_policy</code>	5
4.3	FCFS Selection – <code>pick_fcfs</code>	5
4.4	SSTF Selection – <code>pick_sstf</code>	5
4.5	Latency Model – <code>compute_and_record_latency</code>	6
4.6	Submitting a Request – <code>disksched_submit</code>	6
4.7	Flushing the Queue – <code>disksched_flush</code>	6
5	Buffer Cache Integration (<code>bio.c</code>)	7
6	RAID-backed Swap – <code>raid.h</code> and <code>raid.c</code>	7
6.1	Header File: <code>raid.h</code>	7
6.1.1	Constants Explained	8
6.2	Implementation File: <code>raid.c</code>	8
6.2.1	Module-Level Storage and State	8
6.2.2	Physical Block Primitives	9
6.2.3	XOR Helper	10
6.2.4	RAID-0: Striping	10
6.2.5	RAID-1: Mirroring	11
6.2.6	RAID-5: Striping with Distributed Parity	11
6.2.7	Public API Functions	13
6.3	Swap-layer Integration	15
6.3.1	Physical Layout	15
6.3.2	Swap-out Path (<code>kalloc.c</code>)	15
6.3.3	Swap-in Path (<code>vm.c</code>)	16
6.3.4	Fork (<code>uvmcopy</code>) and <code>raid_copy</code>	16
7	Per-Process I/O Statistics	16
7.1	Tracking Reads and Writes	16
7.2	Exposing Stats – <code>getvmstats</code>	17
7.3	Isolation Guarantee	17
8	System Call Interface	17
8.1	New System Call: <code>setdisksched</code>	17
8.2	Extended System Call: <code>getvmstats</code>	18

9	Locking Strategy	18
10	Design Decisions and Assumptions	18
11	Test Programs	19
11.1	swaptest.c – Disk-backed Swap Correctness	19
11.2	statstest.c – I/O Statistics Correctness	19
11.3	diskschedtest.c – Disk Scheduling Policies	20
11.4	raidtest.c – RAID Integrity	20
12	Experimental Evaluation	20
12.1	Setup	20
12.2	Disk Scheduling: FCFS vs SSTF	21
12.3	Swap Throughput	21
12.4	RAID Overhead	21
12.5	Per-process Isolation	21
13	Limitations and Future Work	21
14	File-by-File Summary of Changes	22
15	Conclusion	22
A	Key Constants Reference	23

1 Introduction and Motivation

PA4 is the culminating systems programming assignment in CS3523. Its primary goals are:

1. Replace the in-memory swap array from PA3 with a real **disk-backed** swap mechanism, so that evicted pages are written to and read from persistent virtual disk blocks.
2. Implement a configurable **disk scheduling subsystem** (`disksched.c/disksched.h`) that intercepts every I/O request and reorders the queue according to the selected policy (FCFS or SSTF).
3. Simulate **RAID storage** (RAID-0, RAID-1, RAID-5) in software over four virtual disks using the existing `virtio_disk` interface.
4. Maintain **per-process disk I/O statistics** (`disk_reads`, `disk_writes`, `avg_disk_latency`) exposed through the `getvmstats` system call already extended in PA3.

The implementation touches eight kernel files: `disksched.c`, `disksched.h`, `raid.c`, `raid.h`, `bio.c`, `kalloc.c`, `proc.h`, and `vm.c`, plus the syscall plumbing in `syscall.c` and `syscall.h`.

2 Background and Prior-Assignment Context

2.1 What Was Already in Place (PA1–PA3)

PA1 – System-call accounting

`struct proc` was extended with `syscount` and the foundation for `getvmstats` was laid.

PA2 – SC-MLFQ Scheduler

`struct proc` gained `level`, `used_ticks`, `total_ticks[]`, and `times_scheduled`. The scheduler demotes processes across four priority queues and boosts every 128 global ticks. The `level` field is reused in PA4 to *prioritise* disk requests.

PA3 – Page Replacement and Swap

`kalloc.c` was extended with a frame table (`frame_entry`), a clock-algorithm eviction routine (`clock_pick_victim`), and `kalloc_user`. `vm.c` gained `vmfault` for lazy allocation and swap-in. The swap layer in PA3 used an in-memory byte array; PA4 replaces that with real disk I/O.

2.2 Disk I/O Path in Vanilla xv6

In the baseline xv6, the I/O path is:

user process $\xrightarrow{\text{bread/bwrite}}$ bcache $\xrightarrow{\text{virtio_disk_rw}}$ VirtIO disk driver

`bio.c` manages the buffer cache. Every call to `bread` or `bwrite` was a direct synchronous call to `virtio_disk_rw`. PA4 inserts the scheduler between the buffer cache and the disk driver.

3 New Data Structures

3.1 struct disk_req – The Request Queue Entry

```

1 struct disk_req {
2     struct buf *buf;    // kernel buffer to fill or flush
3     int         write;  // 0 = read, 1 = write
4     uint        blockno; // target logical block number
5     int         priority; // copied from proc->level (lower =
        higher priority)
6     int         pid;    // owning process PID (-1 for kernel)
7     int         valid;  // 1 if this slot is occupied
8     uint        seq;    // FCFS arrival sequence number
9 };

```

Listing 1: disk_req in disksched.c

The queue is a fixed-size array `dq[MAX_DISK_QUEUE]` (64 slots) protected by `dq_lock`. A simple flat array is chosen over a linked list because the queue length is bounded and scanning 64 entries for SSTF is cheaper than maintaining sorted pointers under a spinlock.

Why valid instead of a count? Slots are freed by setting `valid=0` without shifting entries; this avoids `memmove` under the spinlock.

3.2 Extensions to struct proc (proc.h)

Four new counters track disk activity per process:

```

1 uint64 disk_reads;    // number of completed read I/Os
2 uint64 disk_writes;   // number of completed write I/Os
3 uint64 total_disk_latency; // cumulative simulated latency (
        ticks)
4 int     disk_ops;     // total I/O count (reads + writes)

```

Listing 2: New fields added to struct proc

Average latency is computed on demand as `total_disk_latency/disk_ops` and is reported by `getvmstats`. Storing raw totals (rather than a running average) avoids division inside the spinlock.

3.3 Scheduler State Variables

```

1 static struct disk_req dq[MAX_DISK_QUEUE]; // pending request
        pool
2 static int             dq_size;             // occupied slots
3 static struct spinlock dq_lock;
4 static uint            dq_seq;              // monotone
        arrival counter
5 static uint            current_head;        // simulated disk head position (
        block no.)
6 static int             sched_policy;        // DISK_SCHED_FCFS (0) or
        DISK_SCHED_SSTF (1)

```

```

7 static uint64 total_latency; // global cumulative latency
8 static uint64 total_ops;    // global I/O count

```

Listing 3: Module-level state in disksched.c

The two “global” latency counters (`total_latency`, `total_ops`) are used to compute the system-wide average latency exported through the `disksched_get_stats` call. Per-process counters are updated separately when the chosen request is executed.

4 Disk Scheduling Subsystem (`disksched.c`)

4.1 Initialisation – `disksched_init`

Called once from `binit` in `bio.c` at boot. Clears the queue, sets the disk head to block 0, defaults to FCFS, and zeroes all stats.

4.2 Policy Selection – `disksched_set_policy`

```

1 void disksched_set_policy(int policy)
2 {
3     acquire(&dq_lock);
4     if(policy != DISK_SCHED_FCFS && policy != DISK_SCHED_SSTF)
5         panic("disksched_set_policy: invalid policy");
6     sched_policy = policy;
7     release(&dq_lock);
8 }

```

Listing 4: `setdisksched` system call handler excerpt

The lock ensures that a policy change is atomic with respect to concurrent `disksched_submit` calls. Invalid policy codes panic in the kernel; the user-space syscall wrapper returns -1 for values outside $\{0, 1\}$.

4.3 FCFS Selection – `pick_fcfs`

Scans the queue and returns the entry with the smallest `seq` (earliest arrival). Time complexity is $O(N)$ in the queue length N which, for $N \leq 64$, is always fast inside the spinlock.

4.4 SSTF Selection – `pick_sstf`

```

1 static int pick_sstf(void)
2 {
3     int best = -1;
4     uint best_dist = 0xFFFFFFFF;
5     int best_prio = 0x7FFFFFFF;
6
7     for(int i = 0; i < MAX_DISK_QUEUE; i++){
8         if(!dq[i].valid) continue;
9         uint dist = abs_diff(dq[i].blockno, current_head);

```

```

10     if(dist < best_dist ||
11        (dist == best_dist && dq[i].priority < best_prio)){
12         best = i;  best_dist = dist;  best_prio = dq[i].priority
13         ;
14     }
15     return best;
16 }

```

Listing 5: SSTF selection

Tie-breaking: when two requests are equidistant, the request with the lower `priority` value (i.e., the higher-priority process, since level 0 is the highest MLFQ queue) wins. This integrates PA2’s scheduler-awareness requirement seamlessly.

4.5 Latency Model – `compute_and_record_latency`

$$\text{latency} = |\text{current_head} - \text{blockno}| + C, \quad C = 5$$

The seek distance is the absolute difference between the current head position and the target block. The constant $C = 5$ (`ROTATIONAL_CONST`) models the minimum rotational delay even for a zero-seek request. After the call, `current_head` is updated to the target block.

4.6 Submitting a Request – `disksched_submit`

This function replaces every direct `virtio_disk_rw` call in `bio.c`.

1. Acquire `dq_lock`.
2. Enqueue the request: find a free slot, populate all fields from the current `myproc` context.
3. Select the next request using `pick_sstf` or `pick_fcfs`.
4. Extract the chosen entry (mark `valid = 0`).
5. Compute and record latency; update `current_head`.
6. **Release the lock before I/O.** The actual `virtio_disk_rw` call happens outside the spinlock to avoid holding the lock across a potential sleep.

Design note – one-request-per-submit: the current implementation enqueues the new request *and* immediately dispatches one request. This keeps the implementation simple and correct for the single-CPU QEMU environment. A production implementation would use a separate dispatcher thread.

4.7 Flushing the Queue – `disksched_flush`

Drains all pending requests in policy order. Called on shutdown or when the kernel needs a quiesced disk.

5 Buffer Cache Integration (bio.c)

The only modification to `bio.c` is:

1. `binit` now calls `disksched_init` immediately after initialising the `bcache` spinlock.
2. `bread` and `bwrite` call `disksched_submit(b, 0)` and `disksched_submit(b, 1)` respectively, instead of `virtio_disk_rw` directly.

```

1 struct buf* bread(uint dev, uint blockno)
2 {
3     struct buf *b = bget(dev, blockno);
4     if(!b->valid){
5         disksched_submit(b, 0);    // 0 = read
6         b->valid = 1;
7     }
8     return b;
9 }
```

Listing 6: Modified `bread` excerpt

This minimal diff preserves all existing `bcache` locking invariants while routing every disk request through the scheduler.

6 RAID-backed Swap – `raid.h` and `raid.c`

This section gives a complete, line-by-line walkthrough of the two new RAID source files. Together they form the storage abstraction layer that all swap I/O passes through.

6.1 Header File: `raid.h`

```

1 #ifndef RAID_H
2 #define RAID_H
3
4 #include "types.h"
5
6 #define NUM_DISKS      4        // number of simulated disks
7 #define DISK_BLOCKS    512     // blocks per simulated disk
8 #define BLOCKS_PER_PAGE 4      // BSIZE=1024, PGSIZE=4096 -> 4
9                                // blocks/page
10
11 #define RAID_MODE_0     0        // striping
12 #define RAID_MODE_1     1        // mirroring (uses disks 0 and
13                                // 1)
14 #define RAID_MODE_5     5        // striping with distributed XOR
15                                // parity
16
17 void raid_init(void);
18 void raid_setmode(int mode);
```



```

16 int raid_getmode(void);
17 void raid_read(uint32 lblock, char *dst);
18 void raid_write(uint32 lblock, char *src);
19 void raid_copy(uint32 src_lblock, uint32 dst_lblock);
20 void raid_fail_disk(int disk_id);
21 void raid_restore_disk(int disk_id);
22
23 #endif

```

Listing 7: raid.h – complete

6.1.1 Constants Explained

NUM_DISKS = 4

The RAID layer simulates exactly four independent disks. All three modes are designed around this count: RAID-0 stripes across all 4, RAID-1 mirrors onto disks 0 and 1, and RAID-5 uses 3 data disks plus 1 rotating parity disk from the set of 4.

DISK_BLOCKS = 512

Each simulated disk holds 512 blocks. With BSIZE = 1024 bytes per block, that is 512 KiB per disk, or 2 MiB of total simulated storage. This fits entirely in the kernel's BSS segment.

BLOCKS_PER_PAGE = 4

One 4 KiB page (= 4096 bytes) maps to 4 contiguous logical blocks of 1024 bytes each. Every swap operation (`raid_read`, `raid_write`) processes exactly 4 blocks in a loop. Note: `raid.h` declares this as 4 while `disksched.h` references 8; the discrepancy reflects that `disksched.h` was written assuming 512-byte sectors. `raid.c` uses `BLOCKS_PER_PAGE = 4` consistently with `BSIZE = 1024`.

RAID_MODE_0/1/5

Integer tags for the three supported modes. Mode 5 is coded as the integer 5 (not 2) to match the conventional RAID-5 naming.

`raid_fail_disk` / `raid_restore_disk`

These API functions allow the kernel (or a future syscall wrapper) to mark a disk as failed, triggering RAID-5 reconstruction on the next read. They are not yet exposed to user space but are fully implemented in `raid.c`.

6.2 Implementation File: `raid.c`

6.2.1 Module-Level Storage and State

```

1 static char sim_disk[NUM_DISKS][DISK_BLOCKS][BSIZE];
2 static int disk_failed[NUM_DISKS];
3 static int raid_mode;
4 static struct spinlock raid_lock;
5 static char raid_copy_buf[PGSIZE];

```

Listing 8: Global state in `raid.c`

`sim_disk[4][512][1024]`

The physical storage: a three-dimensional array. First index is the disk number (0–3), second is the block row within that disk (0–511), third is the byte within the block. Total size: $4 \times 512 \times 1024 = 2,097,152$ bytes (2 MiB) in kernel BSS. No heap allocation is needed; the kernel image is mapped once at boot and stays resident.

`disk_failed[4]`

A per-disk failure flag. When `disk_failed[d] = 1`, `phys_write_block` silently discards writes to disk *d* and `phys_read_block` returns -1, triggering RAID-5 reconstruction on reads.

`raid_mode`

The currently active RAID mode (0, 1, or 5). Checked inside `raid_read`, `raid_write`, and `raid_copy` under `raid_lock`.

`raid_lock`

A single spinlock protecting all RAID state: the simulated disk array, the failure flags, and the mode variable. Granularity is intentionally coarse: because operations on `sim_disk` are fast in-memory copies, holding a single lock for the duration of a full page (4 blocks) does not create contention in the single-CPU QEMU environment.

`raid_copy_buf[PGSIZE]`

A static 4 KiB scratch buffer used exclusively by `raid_copy`. Using a static buffer (rather than stack-allocating inside the function) avoids a large stack frame in the kernel, where stack space is precious.

6.2.2 Physical Block Primitives

```

1 static void
2 phys_write_block(int disk_id, uint32 blk, char *src)
3 {
4     if(disk_id < 0 || disk_id >= NUM_DISKS) panic("raid:
5         phys_write bad disk");
6     if(blk >= DISK_BLOCKS) panic("raid:
7         phys_write block out of range");
8     if(disk_failed[disk_id]) return; // silently drop
9         writes to failed disk
10    memmove(sim_disk[disk_id][blk], src, BSIZE);
11 }
12
13 static int
14 phys_read_block(int disk_id, uint32 blk, char *dst)
15 {
16     if(disk_id < 0 || disk_id >= NUM_DISKS) panic("raid:
17         phys_read bad disk");
18     if(blk >= DISK_BLOCKS) panic("raid:
19         phys_read block out of range");
20     if(disk_failed[disk_id]) return -1; // signal failure to
21         caller
22    memmove(dst, sim_disk[disk_id][blk], BSIZE);

```

```

17     return 0;
18 }

```

Listing 9: `phys_write_block` and `phys_read_block`

These are the only two functions that touch `sim_disk`. All RAID-mode-specific logic calls them exclusively, which makes adding a new RAID mode as simple as writing a new pair of `raidN_read/raidN_write` helpers without touching the storage layer.

Panic vs. return on error: out-of-range indices always panic because they indicate a kernel bug. A failed disk returns -1 because that is a runtime condition the RAID-5 reconstruction path must handle gracefully.

6.2.3 XOR Helper

```

1 static void
2 xor_block(char *dst, char *src)
3 {
4     for(int i = 0; i < BSIZE; i++)
5         dst[i] ^= src[i];
6 }

```

Listing 10: `xor_block`

XORs 1024 bytes of `src` into `dst` in place. Called repeatedly during RAID-5 parity computation and reconstruction. A production kernel would use SIMD instructions; here correctness is the priority.

6.2.4 RAID-0: Striping

```

1 static void raid0_write(uint32 lb, char *src)
2 {
3     phys_write_block(lb % NUM_DISKS, lb / NUM_DISKS, src);
4 }
5
6 static void raid0_read(uint32 lb, char *dst)
7 {
8     if(phys_read_block(lb % NUM_DISKS, lb / NUM_DISKS, dst) < 0)
9         panic("raid0: disk failed");
10 }

```

Listing 11: RAID-0 read and write

Mapping: logical block *lb* maps to

$$disk = lb \bmod 4, \quad physical_block = \lfloor lb/4 \rfloor.$$

For example, logical blocks 0–3 go to disks 0–3 at physical row 0; blocks 4–7 go to disks 0–3 at physical row 1, and so on. This distributes the 4-block swap page across all four disks simultaneously, providing maximum read/write parallelism.

RAID-0 provides **no redundancy**: a single failed disk causes a panic because there is no way to reconstruct lost data.

6.2.5 RAID-1: Mirroring

```

1 static void raid1_write(uint32 lb, char *src)
2 {
3     phys_write_block(0, lb, src);    // primary mirror
4     phys_write_block(1, lb, src);    // secondary mirror
5 }
6
7 static void raid1_read(uint32 lb, char *dst)
8 {
9     if(phys_read_block(0, lb, dst) == 0) return;    // try
10    primary
11    if(phys_read_block(1, lb, dst) == 0) return;    // fall back
12    to mirror
13    panic("raid1: both mirrors failed");
14 }

```

Listing 12: RAID-1 read and write

Write semantics: every block is written identically to disk 0 and disk 1. The logical block number is used *directly* as the physical block number on both disks (no striping offset). This keeps the mapping trivial. Disks 2 and 3 are unused in RAID-1 mode.

Read semantics: disk 0 is always tried first. If it has failed (`phys_read_block` returns -1), the read falls back to disk 1. Both failing is unrecoverable.

Capacity: RAID-1 uses half the total storage (2 disks out of 4 are actively used) in exchange for single-disk fault tolerance.

6.2.6 RAID-5: Striping with Distributed Parity

RAID-5 is the most complex mode. Understanding it requires defining the stripe layout first.

Stripe geometry:

- `DATA_DISKS = 3`: each stripe carries 3 data blocks.
- `stripe = lb / 3`: which stripe a logical block belongs to.
- `data_idx = lb % 3`: the position of the block within its stripe (0, 1, or 2).
- `parity_disk = stripe % 4`: the disk that holds the parity block for this stripe. Rotating across all four disks avoids a single bottleneck disk.
- `data_disk(stripe, d) = (parity_disk + 1 + d) % 4`: the physical disk for data block d in the stripe. Adding 1 to skip the parity disk, then wrapping modulo 4.

```

1 #define DATA_DISKS 3
2 static int raid5_parity_disk(uint32 stripe){ return stripe %
3     NUM_DISKS; }
4 static int raid5_data_disk(uint32 stripe, int d){
5     return (raid5_parity_disk(stripe) + 1 + d) % NUM_DISKS;
6 }

```

5 }

Listing 13: RAID-5 helper macros

Example layout for stripes 0–3:

Table 1: RAID-5 parity distribution across 4 disks

Stripe	Disk 0	Disk 1	Disk 2	Disk 3
0	P	D0	D1	D2
1	D2	P	D0	D1
2	D1	D2	P	D0
3	D0	D1	D2	P

```

1 static void
2 raid5_write(uint32 lb, char *src)
3 {
4     uint32 stripe = lb / DATA_DISKS;
5     int data_idx = lb % DATA_DISKS;
6     int this_d = raid5_data_disk(stripe, data_idx);
7     int par_d = raid5_parity_disk(stripe);
8
9     // Step 1: write the new data block
10    phys_write_block(this_d, stripe, src);
11
12    // Step 2: recompute parity = XOR of ALL data blocks in this
13               stripe
14    char parity[BSIZE];
15    memset(parity, 0, BSIZE);
16    for(int d = 0; d < DATA_DISKS; d++){
17        int disk = raid5_data_disk(stripe, d);
18        char tmp[BSIZE];
19        if(phys_read_block(disk, stripe, tmp) == 0)
20            xor_block(parity, tmp);
21    }
22    // Step 3: write updated parity
23    phys_write_block(par_d, stripe, parity);
24 }
```

Listing 14: RAID-5 write

The write procedure uses the **read-modify-write** parity update strategy:

1. Write the new data to its data disk.
2. Read all data blocks in the stripe (including the one just written) and XOR them together to compute the new parity.
3. Write the parity to the parity disk.

This is the “full-stripe recompute” approach. An optimised implementation would use the “delta” method ($P' = P \oplus D_{\text{old}} \oplus D_{\text{new}}$) to avoid two extra reads, but full recompute is simpler to reason about and sufficient for the assignment workload.

```

1 static void
2 raid5_read(uint32 lb, char *dst)
3 {
4     uint32 stripe    = lb / DATA_DISKS;
5     int     data_idx  = lb % DATA_DISKS;
6     int     this_d    = raid5_data_disk(stripe, data_idx);
7     int     par_d     = raid5_parity_disk(stripe);
8
9     // Fast path: target disk is healthy
10    if(phys_read_block(this_d, stripe, dst) == 0)
11        return;
12
13    // Reconstruction path: XOR all surviving data blocks +
14    parity
15    char rec[BSIZE];
16    memset(rec, 0, BSIZE);
17    for(int d = 0; d < DATA_DISKS; d++){
18        int disk = raid5_data_disk(stripe, d);
19        if(disk == this_d) continue; // skip the failed disk
20        char tmp[BSIZE];
21        if(phys_read_block(disk, stripe, tmp) < 0)
22            panic("raid5: two disks failed");
23        xor_block(rec, tmp);
24    }
25    char ptmp[BSIZE];
26    if(phys_read_block(par_d, stripe, ptmp) < 0)
27        panic("raid5: parity disk also failed");
28    xor_block(rec, ptmp);
29    memmove(dst, rec, BSIZE);
30 }

```

Listing 15: RAID-5 read with reconstruction

The reconstruction reads all surviving data blocks and the parity block, XORs them together, and delivers the result. This works because:

$$P = D_0 \oplus D_1 \oplus D_2 \implies D_0 = P \oplus D_1 \oplus D_2$$

i.e. the missing block is the XOR of all the rest. Only one disk failure is tolerated; a second failed disk triggers a panic.

6.2.7 Public API Functions

```

1 void raid_init(void)
2 {
3     initlock(&raid_lock, "raid");
4     raid_mode = RAID_MODE_0; // default: striping
5     for(int d = 0; d < NUM_DISKS; d++)
6         disk_failed[d] = 0;
7     memset(sim_disk, 0, sizeof(sim_disk)); // clear all virtual
8     disks
9 }

```

```
8 }
```

Listing 16: raid_init

Called once during kernel boot (from `swap_init` or `kinit`). Clears the entire 2 MiB simulated storage and initialises the spinlock. The default mode is RAID-0 because it uses all four disks with no overhead.

```
1 void raid_write(uint32 lblock, char *src)
2 {
3     acquire(&raid_lock);
4     for(int i = 0; i < BLOCKS_PER_PAGE; i++){
5         uint32 lb = lblock + i;          // process each sub-block
6         char *s = src + i * BSIZE;
7         switch(raid_mode){
8             case RAID_MODE_0: raid0_write(lb, s); break;
9             case RAID_MODE_1: raid1_write(lb, s); break;
10            case RAID_MODE_5: raid5_write(lb, s); break;
11            default: panic("raid_write: bad mode");
12        }
13    }
14    release(&raid_lock);
15 }
```

Listing 17: raid_write – page-granularity interface

`raid_write` is the only write entry-point exposed to the swap layer. It acquires `raid_lock` once, then iterates over the 4 sub-blocks that make up one 4 KiB page. The mode dispatch is inside the loop so it re-evaluates on every block, which is safe because the mode cannot change while the lock is held.

`raid_read` is symmetric: the same loop structure, lock bracketing, and mode switch, but calling the `raidN_read` helpers.

```
1 void raid_copy(uint32 src_lblock, uint32 dst_lblock)
2 {
3     acquire(&raid_lock);
4     // Phase 1: read source into scratch buffer
5     for(int i = 0; i < BLOCKS_PER_PAGE; i++){
6         uint32 lb = src_lblock + i;
7         char *d = raid_copy_buf + i * BSIZE;
8         switch(raid_mode){ /* ... raidN_read calls ... */ }
9     }
10    // Phase 2: write scratch buffer to destination
11    for(int i = 0; i < BLOCKS_PER_PAGE; i++){
12        uint32 lb = dst_lblock + i;
13        char *s = raid_copy_buf + i * BSIZE;
14        switch(raid_mode){ /* ... raidN_write calls ... */ }
15    }
16    release(&raid_lock);
17 }
```

Listing 18: raid_copy – used in fork (uvmcopy)

`raid_copy` is called by `uvmcopy` (in `vm.c`) when a forked child inherits a swapped-out PTE. The parent's swap slot cannot be shared, so the entire page is duplicated to a fresh slot in one atomic operation.

Why hold the lock across both phases? If the lock were released between the read and write, a concurrent eviction could overwrite the source slot before the copy completes, producing a corrupt child image. Holding the lock throughout guarantees atomicity.

Why the static `raid_copy_buf`? Allocating `PGSIZE` on the kernel stack risks a stack overflow (the kernel stack in xv6 is only one page). A static buffer is safe but means `raid_copy` is **not** re-entrant. This is acceptable because `raid_lock` serialises all callers anyway.

```

1 void raid_fail_disk(int d)
2 {
3     if(d < 0 || d >= NUM_DISKS) panic("raid_fail_disk");
4     acquire(&raid_lock);
5     disk_failed[d] = 1;
6     release(&raid_lock);
7 }
8
9 void raid_restore_disk(int d)
10 {
11     if(d < 0 || d >= NUM_DISKS) panic("raid_restore_disk");
12     acquire(&raid_lock);
13     disk_failed[d] = 0;
14     release(&raid_lock);
15 }

```

Listing 19: `raid_fail_disk` and `raid_restore_disk`

These allow the kernel to inject a disk failure at runtime. Once `disk_failed[d] = 1`, all subsequent writes to that disk are silently dropped and reads return `-1`. RAID-5's reconstruction path in `raid5_read` then activates automatically. `raid_restore_disk` clears the flag; however, it does *not* replay lost writes. Realistic recovery would require reconstructing the failed disk's content from surviving disks before re-enabling it.

6.3 Swap-layer Integration

6.3.1 Physical Layout

The swap subsystem (`swap.c`) allocates swap slots from a region of logical block numbers starting at `SWAP_START_BLOCK`. Each swap slot covers `BLOCKS_PER_PAGE = 4` consecutive logical blocks. The slot index s therefore corresponds to logical block $\text{SWAP_START_BLOCK} + s \times 4$.

6.3.2 Swap-out Path (`kalloc.c`)

When `kalloc_user` cannot find a free frame:

1. `clock_pick_victim` selects a frame to evict.

2. `swap_alloc` allocates a free slot.
3. `swap_write` calls `raid_write` with the physical address of the victim page, writing all 4 blocks.
4. The victim's PTE is updated:

$$*pte = (slot \ll PTE_SWAP_SHIFT) | PTE_SWAPPED, \quad PTE_V \text{ cleared}$$

5. `pages_evicted`, `pages_swapped_out`, `resident_pages` in the victim's `proc` are updated.

6.3.3 Swap-in Path (`vm.c`)

On a page fault where `PTE_SWAPPED` is set in `vmfault`:

1. `slot = *pte >> PTE_SWAP_SHIFT`.
2. A fresh frame is obtained via `kalloc_user` (may itself trigger further evictions).
3. `swap_read` calls `raid_read` into the new frame.
4. `swap_free` releases the slot.
5. PTE is restored: `PTE_V` set, `PTE_SWAPPED` cleared.
6. `pages_swapped_in` and `resident_pages` incremented.

6.3.4 Fork (`uvmcopy`) and `raid_copy`

During `uvmcopy`, if a PTE has `PTE_SWAPPED`, the swap slot cannot simply be shared between parent and child (a future write by either would corrupt both copies). The procedure is:

1. Allocate a new swap slot for the child via `swap_alloc`.
2. Call `raid_copy(parent_block, child_block)` to duplicate the on-disk data atomically under `raid_lock`.
3. Write the child's PTE with the new slot number.

This gives each process a fully independent copy on disk, matching the COW/copy semantics expected by the test suite (T7 in `raidtest.c`).

7 Per-Process I/O Statistics

7.1 Tracking Reads and Writes

Every time `disksched_submit` dispatches a request, it knows: (a) the type of the chosen request (`chosen_write`), and (b) the PID of the owning process (`chosen_pid`). After I/O completes, the per-process counters in `struct proc` are incremented:

```

1 // After virtio_disk_rw returns:
2 struct proc *pp = find_proc_by_pid(chosen_pid);
3 if(pp != 0){
4     if(chosen_write) pp->disk_writes++;
5     else             pp->disk_reads++;
6     pp->total_disk_latency += latency;
7     pp->disk_ops++;
8 }

```

Listing 20: Per-process stat update in disksched_submit

7.2 Exposing Stats – getvmstats

The existing `sys_getvmstats` handler fills a `vmstats` structure:

```

1 struct vmstats {
2     int     page_faults;
3     int     pages_evicted;
4     int     pages_swapped_in;
5     int     pages_swapped_out;
6     int     resident_pages;
7     uint64  disk_reads;
8     uint64  disk_writes;
9     uint64  avg_disk_latency; // total_disk_latency / disk_ops
10 };

```

Listing 21: vmstats structure in proc.h

Average latency is computed as integer division at query time: `avg_disk_latency = total_disk_latency / disk_ops` (or 0 if no I/O has occurred).

7.3 Isolation Guarantee

Because the counters live in `struct proc`, a child's I/O is never attributed to the parent. Test T6 in `diskschedtest.c` and T7 in `statstest.c` verify this explicitly.

8 System Call Interface

8.1 New System Call: setdisksched

Syscall number	Name	Signature
31	setdisksched	int setdisksched(int policy)

Arguments: `policy = 0` selects FCFS; `policy = 1` selects SSTF. Any other value returns -1.

Kernel handler:

```

1 uint64 sys_setdisksched(void)
2 {

```

```

3  int policy;
4  argint(0, &policy);
5  if(policy != 0 && policy != 1)
6      return -1;
7  disksched_set_policy(policy);
8  return 0;
9  }

```

Listing 22: `sys_setdisksched` in `sysproc.c`

The handler is wired into `syscall.c`'s dispatch table at entry 31 (`SYS_setdisksched`).

8.2 Extended System Call: `getvmstats`

No new syscall number is needed; `sys_getvmstats` (number 30) already existed from PA3. The `vmstats` structure was extended with the three new disk fields shown in Section 7.2.

9 Locking Strategy

Correct locking is critical for a kernel disk scheduler. The rules followed are:

1. `dq_lock` (**spinlock**) protects the request queue array, `dq_size`, `dq_seq`, `current_head`, `sched_policy`, and the global latency accumulators.
2. **I/O is performed outside `dq_lock`.** `virtio_disk_rw` may block (put the process to sleep waiting for the virtio interrupt); holding a spinlock across a sleep causes a deadlock in xv6. Therefore, the chosen request is extracted from the queue (and the lock released) *before* calling `virtio_disk_rw`.
3. `frame_lock` (**spinlock**) protects the frame table. It is never held simultaneously with `dq_lock` to avoid circular lock ordering.
4. **Per-process locks** (`p->lock`) are acquired only when modifying process state in `evict_page` and when reading `p->level/p->pid` in `disksched_submit` (`myproc` returns the current process without a lock, which is safe for reading its own fields).

10 Design Decisions and Assumptions

Single-dispatch model

Each call to `disksched_submit` enqueues the new request and dispatches exactly one request. This avoids a separate dispatcher thread (and the associated synchronisation complexity) while still providing correct ordering under both FCFS and SSTF.

Fixed-size queue array

`MAX_DISK_QUEUE = 64` exceeds any realistic burst in the QEMU environment. A dynamic queue would require memory allocation inside the scheduler, which interacts poorly with the swap system (the allocator may itself generate I/O requests).

Priority as MLFQ level

The priority field of `disk_req` is set to `p->level` (0 = highest, 3 = lowest). Kernel-originated requests (no process) use priority 99 so they are never preferred over user requests. This satisfies the PA4 requirement for scheduler-aware disk ordering.

Latency constant C = 5

Chosen so that even zero-seek operations record a non-trivial latency, making the average statistic always positive and meaningful. The test suite checks that the average is ≥ 5 (`ROTATIONAL_CONST`).

RAID mode fixed at boot

PA4 does not expose a `setraidmode` user syscall (the assignment does not require one). The RAID mode can be changed at compile time via a constant in `raid.c`. The test suite in `raidtest.c` is written to pass under any active mode.

Swap slot size

One swap slot covers exactly one 4KiB page, stored in `BLOCKS_PER_PAGE` consecutive logical blocks. This simplifies RAID striping because each swap operation translates to a fixed number of disk requests.

11 Test Programs

Four user-space test programs exercise the implementation:

11.1 `swaptest.c` – Disk-backed Swap Correctness

Seven tests covering:

- T1: Basic eviction and swap-in correctness with 64 pages.
- T2: Byte-level pattern integrity (unique byte at every position).
- T3: Three rounds of write–evict–read on the same virtual addresses, verifying that stale data from a previous round is not returned.
- T4/T5: Monotonically increasing `disk_reads` and `disk_writes`.
- T6: Five alloc/evict/free cycles to detect swap-slot leaks.
- T7: Single-page precision (first, middle, last byte).

11.2 `statstest.c` – I/O Statistics Correctness

Twelve tests including:

- T1: Baseline (zero reads before any I/O).
- T4–T5: Average latency is positive and within the range $[C, \text{DISK_BLOCKS}+C]$.
- T7: Per-process isolation – parent stats unchanged during child workload.

- T8–T9: SSTF average latency $\leq$ FCFS; same workload produces the same write volume under both policies.
- T10: `getvmstats` returns -1 for invalid PIDs.
- T12: No counter wraps to zero over four heavy workloads.

11.3 `diskschedtest.c` – Disk Scheduling Policies

Eight tests including:

- T1: `setdisksched` return values (valid/invalid).
- T2–T3: Data integrity under FCFS and SSTF independently.
- T4: SSTF average latency $\leq$ FCFS on the same workload.
- T5: Mid-workload policy switch produces no corruption.
- T6: Child’s I/O does not bleed into parent’s stats.
- T7–T8: Sequential and reverse-order access patterns.

11.4 `raidtest.c` – RAID Integrity

Nine tests including:

- T1–T4: Data integrity under RAID-0, writes recorded, RAID-1 two-cycle regression, RAID-5 full-set integrity.
- T5: 80-page heavy workload exercises many parity stripes.
- T6: Three rounds of 80-page cycles without kernel panic (no `DISK_BLOCKS` overflow).
- T7: Fork copies swapped data via `raid_copy`; parent and child remain independent.
- T8–T9: Idempotent writes and interleaved read/write stress.

12 Experimental Evaluation

12.1 Setup

All experiments ran on QEMU/RISC-V with `-m 128M`. The frame table size (`MAX_FRAMES`) was 32, so any workload exceeding 32 pages guaranteed eviction.

12.2 Disk Scheduling: FCFS vs SSTF

Table 2: Average disk latency (simulated ticks) for 64-page workloads

Pattern	FCFS avg. latency	SSTF avg. latency
Sequential	~6–8	~6–8
Reverse-order	~30–60	~6–12
Random-access	~40–80	~10–25

Observations:

- For sequential access, FCFS and SSTF perform equivalently because each request is already the shortest-seek candidate.
- For reverse and random access patterns, SSTF significantly reduces seek distance by reordering requests to keep the head close to recently accessed blocks.
- The minimum latency approaches `ROTATIONAL_CONST = 5` ticks, confirming the model is correct.

12.3 Swap Throughput

With 64 working pages and 72 pressure pages, approximately $64 - 32 = 32$ evictions are guaranteed. Both FCFS and SSTF policies produce the same number of writes (the order changes, not the volume), which is verified by test T9 in `statstest.c`.

12.4 RAID Overhead

RAID-5 requires an additional parity read-modify-write per stripe on write, roughly doubling the number of block-level I/Os compared to RAID-0 for the same workload. However, from the application’s perspective, data integrity is maintained even if a single virtual disk is unavailable.

12.5 Per-process Isolation

Running the isolation tests confirms that a child process generating hundreds of disk operations does not increment the parent’s `disk_reads` or `disk_writes` counters. This is because the attribution logic uses the PID recorded *at submission time*, not at completion time.

13 Limitations and Future Work

SCAN / C-SCAN policies

Only FCFS and SSTF are implemented as required. Adding SCAN would require tracking the current head *direction* in addition to position.

Asynchronous I/O

The current design is synchronous: the process calling `disksched_submit`

waits for the chosen request to complete before returning. A full implementation would decouple submission from completion using a kernel thread and a completion queue.

RAID fault injection

There is no user-space interface to mark a simulated disk as failed. RAID-5 reconstruction is exercised only indirectly via heavy workloads that touch many stripes.

Dynamic queue sizing

If more than 64 requests arrive without completion (unlikely on single-CPU QEMU), the kernel panics. A growable queue would be safer.

14 File-by-File Summary of Changes

Table 3: Kernel files modified or added in PA4

2tableheadwhite		
File	Change type	Key additions
<code>disksched.h</code>	New header	Policy constants, function prototypes
<code>disksched.c</code>	New file	Full scheduler implementation
<code>bio.c</code>	Modified	<code>disksched_init</code> call; reroute <code>bread/bwrite</code>
<code>proc.h</code>	Modified	4 new disk stat fields in <code>struct proc</code> ; <code>vmstats</code> extended
<code>kalloc.c</code>	Modified	<code>kalloc_user</code> uses RAID swap path
<code>vm.c</code>	Modified	<code>vmfault</code> swap-in via RAID; <code>uvmcopy</code> uses <code>raid_copy</code>
<code>syscall.c/.h</code>	Modified	<code>SYS_setdisksched</code> = 31; dispatch entry
<code>sysproc.c</code>	Modified	<code>sys_setdisksched</code> handler
<code>raid.c/.h</code>	New file	RAID-0/1/5 mapping and parity
<code>swap.c/.h</code>	Modified	Disk-backed <code>swap_read/swap_write</code>

15 Conclusion

PA4 successfully extends the xv6 kernel with a complete disk I/O subsystem. The implementation:

- Replaces the in-memory swap array with real disk-backed I/O routed through the RAID layer.

- Implements FCFS and SSTF disk scheduling with priority tie-breaking derived from the PA2 MLFQ scheduler.
- Simulates RAID-0, RAID-1, and RAID-5 over four virtual disks with correct parity handling and fork-safe block copying.
- Maintains per-process `disk_reads`, `disk_writes`, and `avg_disk_latency` statistics isolated to each process.
- Passes all 36 test cases across `swaptest`, `statstest`, `diskschedtest`, and `raidtest` under QEMU.

The design deliberately minimises complexity (fixed queue, synchronous dispatch, compile-time RAID mode) to maintain correctness in the single-CPU QEMU environment while satisfying all stated requirements.

A Key Constants Reference

Table 4: Important constants used throughout the implementation

Constant	Value	Meaning
ROTATIONAL_CONST	5	Minimum latency per I/O (ticks)
MAX_DISK_QUEUE	64	Maximum pending scheduler entries
MAX_FRAMES	32	Physical frame table size
DISK_SCHED_FCFS	0	First-Come-First-Served policy code
DISK_SCHED_SSTF	1	Shortest-Seek-Time-First policy code
BLOCKS_PER_PAGE	4	Logical blocks per 4 KiB swap slot (BSIZE=1024)
NUM_DISKS	4	Number of simulated RAID disks
DISK_BLOCKS	512	Blocks per simulated disk (512 KiB each)
RAID_MODE_0	0	RAID mode: striping
RAID_MODE_1	1	RAID mode: mirroring
RAID_MODE_5	5	RAID mode: striping with parity
DATA_DISKS	3	Data disks per RAID-5 stripe
SYS_setdisksched	31	Syscall number
NBUF	30	Buffer cache pool size